Computers & Security 154 (2025) 104399

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Penterep: Comprehensive penetration testing with adaptable interactive checklists

Willi Lazarov [a,*], Pavel Seda [a], Zdenek Martinasek [a], Roman Kummel [b]

[a] Brno University of Technology, Antonínská 548/1, Brno 601 90, Czechia
[b] HACKER Consulting, Luběnice 65, Luběnice 783 46, Czechia

ARTICLE INFO

Keywords:
Checklists
Cybersecurity
Ethical hacking
Methodology
Penetration testing
Reporting
Vulnerability assessment

ABSTRACT

In the contemporary landscape of cybersecurity, the importance of effective penetration testing is underscored by NIS2, emphasizing the need to assess and demonstrate cyber resilience. This paper introduces an innovative approach to penetration testing that employs interactive checklists, supporting both manual and automated tests, as demonstrated within the Penterep environment. These checklists, functioning as a quantifiable measure of test completeness, guide pentesters through methodological testing, addressing the inherent challenges of the security testing domain. While some may perceive a limitation in the dependency on predefined checklists, the results from a presented case study underscore the criticality of methodological testing. The study reveals that relying solely on fully automated tools would be inadequate to identify all vulnerabilities and flaws without the inclusion of manual tests. Our innovative approach complements established methodologies, such as PTES, OWASP, and NIST, providing crucial support to penetration testers and ensuring a comprehensive testing process. Implemented within the Penterep environment, our approach is designed with deployment flexibility (both on-premises and cloud-based), setting it apart through an overview comparison with existing tools aligned with state-of-the-art penetration testing approaches. This flexible and scalable approach effectively bridges the gap between manual and automated testing, meeting the increasing demands for effectiveness and adaptability in penetration testing.

1. Introduction

Penetration testing, short pentesting, is growing in importance and is becoming a necessary part of every company or organization that uses information and communication technologies (ICT). The ever-increasing threat of cyberattacks and known cyberattack cases emerging in critical situations such as cyberattacks on hospitals during the COVID-19 pandemic (Muthuppalaniappan and Stevenson, 2020; Wasserman and Wasserman, 2022; Ahmed et al., 2023) support this claim (Sharif and Mohammed, 2022). Furthermore, cyberattackers use more complex and sophisticated techniques to exploit and use system vulnerabilities. Next, organizations due to the increasing integration of information technologies into their regular processes (e.g., in hospitals (Tadiboina, 2022) or banking sector (Shulha et al., 2022)) become more exposed to risks associated with cyberattacks. Penetration testing allows companies and individuals to simulate real-world attacks and identify weaknesses in their systems. Using the results from penetration testing, companies can react quickly and strengthen their cyber resilience. Next, regular penetration testing is appropriate due to the dynamic increase of new technologies such as the Internet of Things (IoT) (Heiding et al., 2023; Bella et al., 2023), artificial intelligence (AI) (Devagiri et al., 2022), or cloud computing (Jayadeva et al., 2023) that enable new cyber threats (El Kafhali et al., 2022).

The increasing demand to conduct regular penetration tests is also highlighted by the European Directive Network and Information Security (NIS2), which expands the number of obliged subjects and the scope of responsibilities, such as the need to test and demonstrate cyber resilience (European Union, 2022). Furthermore, there are several publications provided by NIST (2020) (e.g., 800-53) providing the catalog of security and privacy controls for information systems and organizations to protect organizational operations and assets, individuals, other organizations, and the nation from a diverse set of threats and risks, including hostile attacks, human errors, natural disasters, structural failures, foreign intelligence entities, and privacy risks. All of these recommendations aim to increase the organization's protection of sensitive

* Corresponding author.
E-mail addresses: lazarov@vut.cz (W. Lazarov), sedap@vut.cz (P. Seda), martinasek@vut.cz (Z. Martinasek), kummel.r@penterep.com (R. Kummel).
URLs: https://www.vut.cz/en/people/willi-lazarov-221556 (W. Lazarov), https://www.vut.cz/en/people/pavel-seda-154208 (P. Seda), https://www.vut.cz/en/people/zdenek-martinasek-88846 (Z. Martinasek).

https://doi.org/10.1016/j.cose.2025.104399
Received 20 February 2024; Received in revised form 20 February 2025; Accepted 20 February 2025
Available online 17 March 2025
0167-4048/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

data and information, ensure regulatory compliance, and minimize the company's or organization's risk of financial and reputational losses associated with cyber incidents (Teichmann and Boticiu, 2023).

Penetration testing is performed using various testing tools that identify vulnerabilities, provide security assessment, assist in risk mitigation, provide security assurance, help improve incident response plans, assess the security of web applications (vulnerabilities such as SQL injection or cross-site scripting), improve security awareness by notifying employees about potential security threats, and help assess the security of third-party vendor services or solutions (Dalalana Bertoglio and Zorzo, 2017). The representative set of available tools can range from scanner tools such as Nmap (NMAP.org, 2025) or Metasploit (RAPID metasploit, 2025) to complex solutions such as Burp Suite (PortSwigger, 2025) or Zed Attack Proxy (ZAP) (ZAP, 2025). A detailed overview of tools to support penetration testing is provided in Section 2.2.

1.1. Problem statement

The penetration testing varies depending on the methodologies used and the environment being tested, making it challenging to generalize the testing procedure (Denis et al., 2016). For instance, testing network infrastructure differs significantly from web application penetration testing. When examining a computer network, the tester will work mainly with the IP address, subnet mask, and default gateway of the network, identifying different types of network and endpoint devices (e.g., router, server, firewall, etc.) and assorted network services with different parameters. On the other hand, web application penetration testing might involve identifying various programming languages, frameworks, and other web pages utilized with various user inputs. Due to these variations, it is not feasible to create a one-size-fits-all procedure for every potential test target. Instead, different ICT domains require individualized approaches, adding a layer of complexity to penetration testing.

In an effort to mitigate the above-mentioned penetration testing challenges, a number of automated tools have been developed. However, these automated testing tools do not ensure the completeness of the penetration testing which is critical. This limitation arises because certain vulnerabilities are either hard or even impossible to test automatically, particularly those that appear in various parts of the tested environment, represent business logic errors, or necessitate visual inspection. However, given the natural purpose of penetration testing, it is necessary to detect all vulnerabilities within a specific scope. Therefore, a suitable combination of manual and automated testing based on a measurable methodology (e.g., proposed interactive checklist) is necessary. To the best of our knowledge, current tools and applications fall short of offering this possibility.

1.2. Key contributions

In this section, we highlight the key contributions of our work, providing a unique perspective on effective, transparent, and complete penetration testing of the tested targets. We denote the following key contributions of this paper:

- An innovative approach to penetration testing that effectively supports both automated and manual tests.
- Design, implementation, and verification of the innovative approach in the developed Penterep environment.
- Showcase using the provided case study, concluded with lessons learned and penetration testing best practices.

Based on the problem outlined in Section 1.1, we propose an innovative approach to penetration testing that combines manual and automated testing while ensuring the completeness, transparency, and accuracy of penetration testing. To apply and verify our innovative approach, we developed the Penterep environment. The Penterep provides interactive checklists, a knowledge base containing predefined tests and vulnerabilities, as well as support for automated testing or report generation.

Penterep helps testers perform automated and manual tests with less probability of missing any important test. Additionally, it enables the storage of test records, reports, and results, which can be easily shared between testers. Among the application of standard methodologies such as the Open Worldwide Application Security Project (OWASP), Open Source Security Testing Methodology Manual (OSSTMM), ISSAF (Information Systems Security Assessment Framework), and Penetration Testing Execution Standard (PTES), testers can complement the methodology with their checklists for a variety of tests and vulnerabilities. In addition to other solutions, Penterep allows the personalization of test selection for specific targets based on its properties. The very appreciated capability is also the possibility to deploy the Penterep environment on-premise or within the cloud, which increases the usability of the solution for various customers and their specific requirements. We demonstrate the environment's capabilities using a case study showing exploitation on selected web pages and comparing it to other state-of-the-art existing tools or other solutions.

1.3. Paper organization

This paper is organized into six sections. The background and related work are summarized in Section 2. Section 3 provides the platform capabilities and the ability to create its methodologies on top of the standard ones. Section 4 describes the Penterep architecture, data collection, deployment use cases, available workflows to conduct the penetration test, and the usage perspective of the pentester and the customer who requests the pentest. Section 5 presents a case study showing the results of penetration testing using Penterep on the selected web application. In this section, the lessons learned from the Penterep deployment are described for the selected scenarios. Section 6 concludes the paper and introduces open challenges.

2. Background and related work

Penetration testing is an extensive method of assessing the security level of information and communication systems. Testing can be related to network infrastructure, services, application layer protocols, web applications, or end users. The main objective of penetration testing is to preventively detect, verify, and describe vulnerabilities of the tested target so that all vulnerabilities found can be remediated. This method of security testing falls under the realm of ethical hacking and is done quite legitimately with the prior consent of the tested subject, most often by an external party under contract. The scope and process of penetration testing may vary depending on the methodology, the environment being tested, and the way it is performed (Vats et al., 2020; Nidhra and Dondeti, 2012; Bacudio et al., 2011). Methodologies determine how to proceed with penetration testing. Commonly used methodologies include OWASP, NIST, OSSTMM, and PTES. The different phases of penetration testing are shown in Fig. 1 (Vats et al., 2020; Bacudio et al., 2011).

The first phase of penetration testing is *Pre-engagement*: Before starting the test, it is important to determine the scope, logistics, rules, and all objectives of the penetration test based on financial and time capacity (Rak et al., 2022). *Information gathering*: This phase aims to obtain as much information as possible about the environment under test, such as the network infrastructure, devices, services, or technologies used, and their parameters (e.g., operating system, software, and its version) (McKinnel et al., 2019). *Vulnerability assessment*: Based on the information gathered in the previous testing phase, the vulnerability discovery process includes two main parts: searching

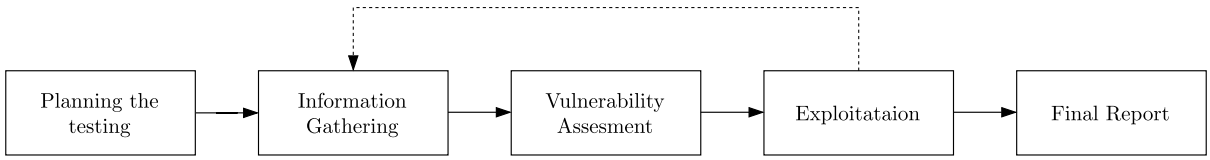


Fig. 1. Penetration testing process overview.

for vulnerabilities in databases of known vulnerabilities[1] and partial testing of the environment for the presence of a specific vulnerability (e.g., injection vulnerability, misconfiguration, etc.) (Blakley and Cranor, 2023). *Exploitation*: Discovering vulnerabilities as a result of penetration testing is not enough. The risks found must be verified by exploiting the vulnerability. The exploitation of a vulnerability may reveal a previously inaccessible part of the tested environment (e.g., the administration of a web application). When such a case happens, the previous two phases have to be repeated (Bacudio et al., 2011). *Final report*: The final report describes the identified risks, determines their severity, and makes recommendations based on which the contracting authority should take appropriate action (e.g., fix source code, system updates, etc.) (Alghamdi, 2021).

2.1. Penetration testing approaches

Penetration testing can be assessed: (i) according to the pentester starting position, (ii) by the level of knowledge of the environment, (iii) by the method execution (Scarfone et al., 2008; Denis et al., 2016). According to the *tester starting position*, penetration testing can be divided into (Xie et al., 2022; Gangupantulu et al., 2022):

- **External testing**: that is performed without authorization and access to an internal part of the tested environment. According to the phases listed in Fig. 1, the tester attempts to compromise the target and obtain or modify protected assets. Therefore, the main objective of external testing is to test the level of security against threats from the external environment of the tested target (usually the Internet) (Gangupantulu et al., 2022).
- **Internal testing**: where the tester has access to the internal part of the tested environment. In this case, the tester represents a legitimate user (e.g., an employee of the tested entity) or an attacker who accesses the intranet. In internal testing, security is evaluated, in particular, security mechanisms, system policies, and access control (Xie et al., 2022; Gangupantulu et al., 2022).

By the *level of knowledge of the environment*, penetration tests can be further divided into (Aydos et al., 2022; Heiding et al., 2023; Rawal et al., 2023):

- **Black-box tests**: performed without initial knowledge of the tested environment. Testing consists of gradually uncovering individual components and finding their vulnerabilities. An example of such a vulnerability could be an outdated version of an operating system, a misconfigured web server, or an untreated user input of the application under test (Zhu et al., 2022; Aydos et al., 2022; Heiding et al., 2023).
- **White-box tests**: which rely on complete knowledge of the test environment. The tester has full access to all system components, source code, database, etc. For white-box tests, a deeper analysis is usually performed. In the case of application testing, there is also a static analysis during which the tester looks for security vulnerabilities directly in the source code (Heiding et al., 2023; Rawal et al., 2023).
- **Gray-box tests**: which are composed of a combination of white-box and black-box tests. To reduce time consumption, the tester has information to facilitate finding and verification of vulnerabilities in the environment under test (e.g., source code, structure database, or access data). Unlike white-box tests, there is no complete static analysis of the source code. Information tends to be used only to better understand the functionality of the environment under test as opposed to black-box tests, where the tester must obtain this information himself (Heiding et al., 2023; Trickel et al., 2023; Aydos et al., 2022).

By the *method execution*, penetration testing can be divided into (Auricchio et al., 2022; Mishra, 2021; Bartusiak et al., 2023):

- **Manual testing**: where the penetration tester has full control over the testing process and the results obtained are completely dependent on his process and skills. The advantage of manual testing is, in particular, the accuracy and consistency in the detection of vulnerabilities. On the other hand, the disadvantage of this method is time consumption and the need to have extensive knowledge (Mishra, 2021).
- **Automated testing**: using automated tools to find vulnerabilities in a tested environment. The tools use a set of predefined test strings, databases of publicly disclosed vulnerabilities, etc. The tester is then presented with the results of the possible findings. Compared to manual testing, this method is much faster and requires only knowledge of how to work with the tools. On the other hand, the disadvantage may be a lower success rate in finding vulnerabilities, false positives, or the inability to test for some vulnerabilities (e.g., when the vulnerability manifests itself in a different location than the input or logical reasoning is required) (Auricchio et al., 2022).
- **Semi-automated testing**: which combines manual and automated tests, using the advantages and mitigating the disadvantages of both types. The exploration and identification phase is carried out primarily by automated tools, while potential vulnerabilities and the possibility of exploitability are tested manually (Bartusiak et al., 2023).

2.2. Tools overview

There are many different penetration testing tools on the market, which can be further divided according to the target area (e.g., computer networks, web applications, mobile applications, etc.). Automated penetration testing tools, such as Nmap (NMAP.org, 2025), Metasploit (RAPID metasploit, 2025), SQLmap (SQLmap, 2024), or Nikto (CIRT.net, 2024), represent some of the widely used tools. On the other hand, solutions for management alone, support for manual penetration testing, or a combination of both are less numerous groups. Since our solution is comprehensive and aims to support all the operations performed during penetration testing, we make the overview comparison exclusively against platforms and applications that are not purpose-built and represent this less numerous group of tools. The comparison is listed in Table 1 and includes the following solutions:

- **Burp Suite**: Software tool for web application penetration testing. The tool is distributed in 3 editions – Community, Pro, and Enterprise. Although the community edition primarily supports manual tests that can be customized, the Pro and Enterprise editions offer fully automated web vulnerability scans (PortSwigger, 2025).

[1] Common Vulnerabilities and Exposures (CVE)

Table 1
Feature overview of the selected penetration testing tools compared to the proposed solution.

Feature/Functionality	Burp Suite	Acunetix	Nessus	w4af	Wapiti	Penterep
Support for automatic testing	✓	✓	✓	✓	✓	✓
Support for manual testing	✓	✗	✗	✗	✗	✓
Web application testing	✓	✓	✓	✓	✓	✓
Mobile application testing	✗	✗	✗	✗	✗	✗
Network infrastructure testing	✗	✗	✓	✗	✗	✓
Application server testing	✗	✗	✓	✗	✗	✓
Periodic testing (scheduling)	✓	✓	✓	✗	✗	✗
Detailed test settings	✓	✗	✗	✗	✗	✓
Environment visualization	✗	✗	✗	✓	✗	✓
Interactive checklists	✗	✗	✗	✗	✗	✓
Documentation and help	✓	✓	✓	✓	✓	✓
Records of findings	✓	✓	✓	✗	✓	✓
Severity indicator	✓	✓	✓	✓	✓	✓
Severity calculator	✗	✗	✓	✗	✗	✓
Graphical user interface	✓	✓	✓	✓	✗	✓
Mobile device support	✗	✓	✓	✗	✗	✓
Import of results	✗	✗	✗	✗	✗	✓
Export of results	✓	✓	✓	✓	✓	✓
Project management	✗	✗	✓	✗	✗	✓
Custom report templates	✗	✓	✗	✗	✗	✓
Multi-user support	✗	✓	✓	✗	✗	✓
Free license version	✓	✗	✗	✓	✓	✓
Integration of custom tools	✓	✓	✓	✓	✗	✓
Third party integration	✓	✓	✓	✓	✗	✗

- **Acunetix**: Web application for continuous vulnerability scanning and testing. The main features of this solution include more than 7000 vulnerability scans, an easy-to-use user interface, and various integrations. The Acunetix application is only offered in annual subscription mode with hidden pricing (Acunetix, 2024).
- **Nessus**: Automated vulnerability assessment tool for web application, cloud, and external attack surface scanning. Nessus can be configured and extended with community or custom plugins and is distributed in two commercial editions — Expert and Professional (Tenable, 2024).
- **w4af**: Open source framework for detecting and exploiting vulnerabilities. The framework is based on the previous community-driven w3af project. The new version offers dockerization, all plugins from its predecessor, and compatibility with Python 3. The w4af plugins are configurable and can be used for crawling, auditing and exploiting vulnerabilities (W4af, 2024).
- **Wapiti**: Open source vulnerability scanner. The scanner includes a set of tests, including fuzzing, injecting payloads, folder and file enumeration, searching for backups, and other automated tasks (GitHub, 2024).
- **Penterep**: Web platform for penetration testing (the solution presented in this paper). The Penterep platform supports both manual and automated testing and aims to cover the whole testing area and include all steps of the penetration test.

The comparison of Penterep (the presented solution) with Burp Suite, Acunetix, Nessus, w4af, and Wapiti is shown in Table 1. The Penterep platform addresses the issue of penetration testing to maximize testing capabilities, supports manual testing, and increases overall coverage of the tested area. This makes it significantly different from other tools that specialize only in specific ICT subareas. This paper approaches the problem with an innovative approach to address the shortcomings of other solutions with the main goal of providing complete support for penetration testing.

Apart from Penterep and Burp Suite, the other compared tools are limited to automated testing. One such tool is w4af, which allows for more configurable automated tests but lacks the ability to manage projects, and its scope of use for penetration testing is therefore limited. The Acunetix web application, Nessus, and Wapiti build testing primarily on its scanners, with little or no configuration and a preview of the entire automated testing process. Due to this limitation, they are used more for vulnerability assessment than as a tool to support manual penetration testing. Burp Suite, on the other hand, offers advanced automated testing capabilities through its modules, including its configuration and integration of community extensions, but its use is limited to penetration testing of web applications.

It should be noted that these are long-developed tools. The oldest of all is Nessus, which was developed in 1998 under the brand name "The Nessus Project" (Carey et al., 2008). Burp Suite was developed in 2003 (PortSwigger, 2025) and Acunetix has been on the market since 2005 (Acunetix, 2024), both commercial solutions for large companies. The w4af tool has been developed since 2013 as an open source tool and is therefore community-driven (W4af, 2024). Similarly, the open source tool Wapiti is being developed by the community since 2006 (GitHub, 2024). The actual development of the Penterep web platform started in May 2021 and the first Proof of Concept (PoC) was completed in June of the same year. Thanks to its architecture, modularity, and the methodology used, it was possible to create a large-scale environment for penetration testing in a short time.

3. Innovative pentesting methodology

Different methodologies for penetration testing define the phases that testers follow. Most of them are generic and differ mainly in their steps. Among the more well-known, these include OSSTMM (1. Induction Phase, 2. Interaction Phase, 3. Inquest Phase, 4. Intervention Phase) (ISECOM, 2024), PTES (1. Pre-engagement Interactions, 2. Intelligence Gathering, 3. Threat Modeling, 4. Vulnerability Analysis, 5. Exploitation, 6. Post Exploitation, 7. Reporting) (PTES, 2024), or ISSAF (1. Planning and Preparation, 2. Assessment, 3. Reporting) (Rathore et al., 2024). However, these methodologies are no longer up-to-date, with PTES not having been updated since 2014, OSSTMM since 2010 and ISSAF since 2006.

Among the current and widespread methodologies and frameworks are documents from NIST (Special Publication 800-115) (Scarfone et al., 2008), OWASP (4 standards) (OWASP Foundation, 2024b), and ISO 27001 (ISO, 2024). While OWASP standards are primarily specialized in web and mobile applications, NIST covers practical testing methodology, and ISO/IEC 27001 is the standard for information security management systems (ISMS).

Our methodology is based on adaptable interactive checklists. Each checklist contains a specific set of test cases based on an existing open source standard (see Fig. 2), such as the OWASP Application Security Verification (ASVS) (OWASP Foundation, 2024a), NIST SP 800-63B or

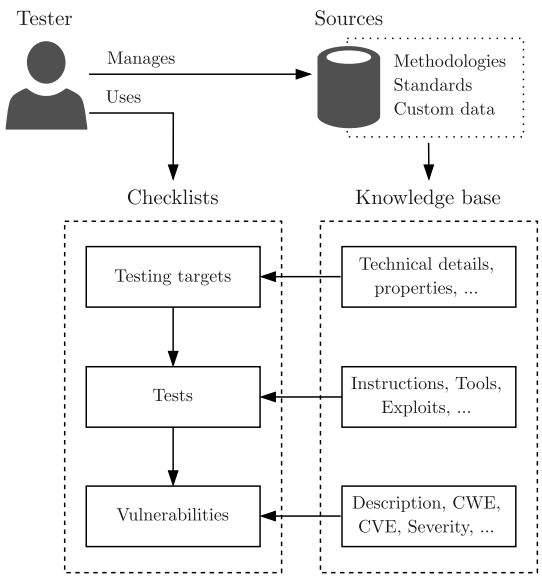


Fig. 2. Structure of interactive checklists.

on testers' custom checklists. Each test contains the following properties: name, objective, description, and instructions on how to perform the test. The tests are further divided into groups (e.g., authentication, authorization, session management, etc.) and by type of test objective. Another added value is the assignment of possible vulnerabilities and other weaknesses that can be found by the selected test. The interactive checklists are adaptive, and therefore our methodology presents testers only with the relevant penetration tests to increase accuracy and reduce the overall testing time. An example of a checklist-based penetration test is provided in Appendix C.

Penetration tests are adapted based on the selected checklist, the test target, and its properties. For example, if a tester finds a web page with an HTML form, it is presented with subtests on the form inputs. If the input is a "string" type, 4 specific tests are presented, but if it is a "file upload", other tests are presented for that target type. Conversely, a web resource of a type text file (e.g., robots.txt, readme.txt, etc.) may be found where there are no forms, but it is tested to see if the file contains, for example, sensitive information or paths that should not be accessible by the user. The methodology also works with an environment where tests' target of a specific system (e.g., Linux) will not be submitted when testing another system (e.g., Windows).

Since penetration tests are designed to find and verify various vulnerabilities, one or more vulnerabilities are assigned to each test as part of our methodology. Each vulnerability contains predefined code, name, description, cause, impact, recommendation, and severity. In this way, the tester does not have to manually write everything down when the test is finished and the relevant vulnerabilities are already present. Once a vulnerability is selected, each of the listed properties can be further customized, for example, to further describe the manifestations of the vulnerability in the context of the tested target, to indicate the exact location of the vulnerability, to recalculate the severity, etc. Like custom tests, the vulnerability can be completely customized by the tester independently of the selected checklist.

Another crucial aspect of our approach involves the integration of different checklists. The penetration testing process is contingent on its scope, allowing for the testing of individual applications or extensive infrastructures comprising various network devices and application servers. We assign a set of targets to each checklist, and these targets undergo specific tests based on their properties. For instance, consider the following procedure: (i) utilizing the checklist for testing infrastructure, the tester scans the network to identify communicating devices; (ii) selecting a device, the tester performs a test, revealing it as a server; (iii) following the infrastructure checklist, the tester scans open ports, such as port 80 for web services; (iv) upon discovering an application on the web service, the tester applies the checklist for web application penetration testing, in conjunction with the checklist for testing infrastructure, triggering another set of tests. Alternatively, when the objective is to perform penetration testing solely on a specific web application, the test target is predefined as that particular web application. In this scenario, the tester uses only the checklist for web application penetration testing without the checklist for infrastructure testing as in the previous scenario. This interactive testing approach provides flexibility for testers based on the conditions and scope of penetration testing.

4. Penterep environment

To support and increase the efficiency of manual and automated penetration testing, we have developed the Penterep penetration testing environment. Penterep is a web penetration testing platform that guides the penetration tester through interactive checklists so that it is always clear what test is about to be performed. For each test, the tester is also provided with detailed instructions on how to run the test, making it possible to involve less experienced testers who are learning simultaneously during testing. Any findings are recorded in the system as the results of individual tests, including severity and attachments.

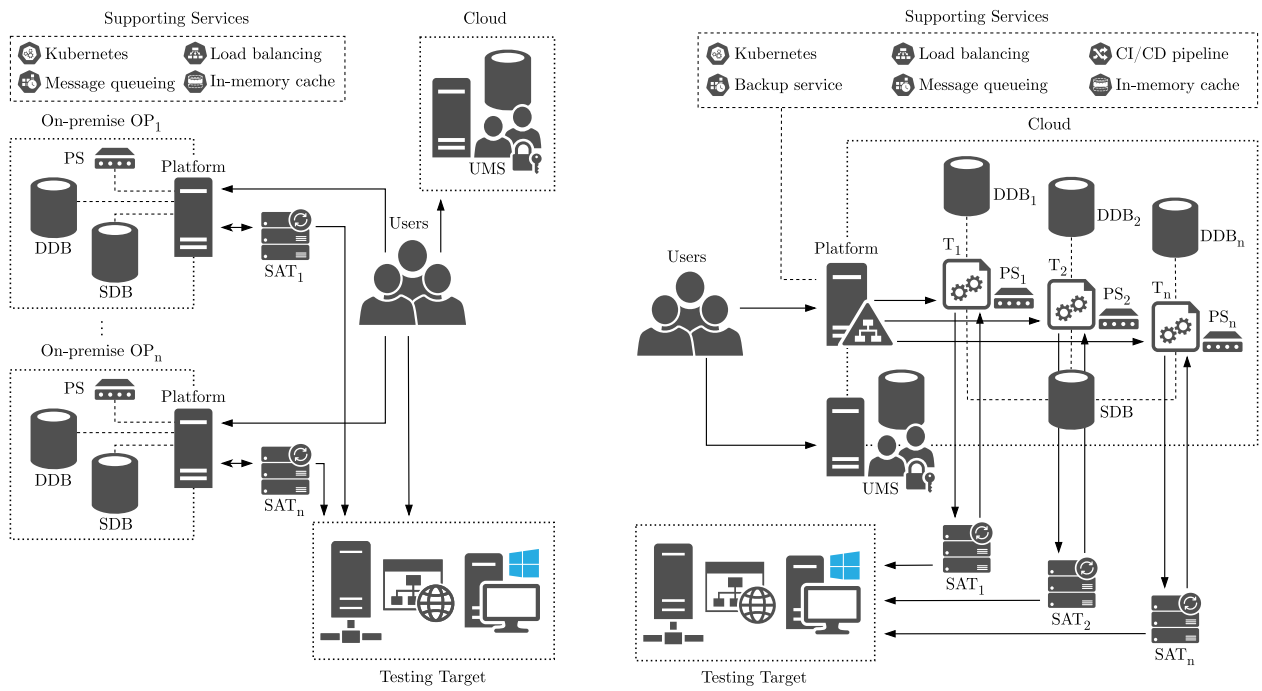
The penetration testing environment provides an extensive knowledge base in which all vulnerabilities are already thoroughly described or can be added as needed. Testers then only need to select a specific vulnerability from the list. At the same time, multiple testers can work in the testing environment without leaving any targets untested or duplicating the work of other testers. This is because each team member has an immediate overview of which tests have been completed and which are still waiting to be executed. In addition, testers also have the option for selected test cases to call automated tests directly from Penterep, with results being fed back after the work is completed, where they are automatically recorded in the system.

When the penetration test is complete, Penterep produces a fully automated report listing all the details and a list of all the vulnerabilities discovered, including their severity, recommendations for remediation, etc. Thanks to the knowledge base, links to references, descriptions of relevant tests, and other information are also added to the report for each finding. Any attachments uploaded by testers during testing are also automatically included in the report.

4.1. Penterep architecture

The environment for penetration testing supports both on-premise (self-hosted) and cloud-based Software as a Service (SaaS) solution for individual deployment. The complete production environment, including all its components, can be seen in Fig. 3. The Penterep system in both use cases (on-premise and cloud) consists of dynamic database (DDB), static databases (SDB), persistent storage (PS), servers for automated testing (SAT), the platform itself, and user management system (UMS).

The SDB contains the initial setup, test cases and a knowledge base with possible penetration testing target types, their characteristics, vulnerabilities, manual test procedures, guidance on testing those vulnerabilities and, where applicable, parameterized commands to run automated tests against specific targets using the appropriate tools. For example, if the user selects testing based on the OWASP ASVS checklist, the SDB provides all the tests outlined in ASVS. The SDB is regularly updated to remain up-to-date and can be further extended with new checklists (e.g., for another domain like IoT or cloud penetration testing). Each project for penetration testing is then stored in the DDB and the SDB data are accessed in read-only mode.



(a) Penterep on-premise deployment

(b) Penterep cloud deployment

Fig. 3. Penterep architecture consists of Users; Cloud platform environment with Tenants T_1, T_2,..., T_n; Dynamic databases DDB_1, DDB_2,..., DDB_n; Persistent storages PS_1, PS_2 ..., PS_n; Static database SDB; User Management System (UMS); Servers for automated testing SAT_1, SAT_2,..., SAT_n; On-premise platforms OP_1, OP_2,..., OP_n; and Testing Target.

As noted, the DDB contains data from real penetration testing (IP addresses, URLs, files, found vulnerabilities, reports, etc.) highlighting the results from the penetration testing. Therefore, working with dynamic data is entirely separate from the definition of penetration testing use cases in the static database. Persistent storage is a separate database that efficiently stores files and other binary data often generated during penetration testing.

The next key component is the Server for automated testing (SAT). Users either perform testing manually and upload the results to the Penterep or run automated tests that are executed by the SAT that saves the results to the Penterep when the test is complete. The target of an automated test can be almost anything, as long as the SAT has a suitable built-in tool to test it. There can be multiple test servers, and the SAT can also run on the user's device. The Penterep, to support penetration testing as much as possible, allows testing to be performed not only by penetration testers, but also by other users, such as developers, who fix the found vulnerabilities, and the tester then retests the presence of the given vulnerability. The key feature of SAT is the dividing automation process into predefined single-purpose subtests that increase the accuracy of the overall penetration testing and therefore reduce false positives.

In the cloud, we have to separate the actual results of penetration testing from the adaptable interactive checklists provided by the platform. For that reason, we design the solution as a multi-tenant system, where each tenant T_1, T_2,..., T_n accesses a logically separate application connected to the tenant's dynamic database DDB_1, DDB_2,..., DDB_n and persistent storage PS_1, PS_2,..., PS_n. While each present tenant has an assigned dynamic database and persistent storage with write privileges, the static database SDB is shared with the others and is kept read-only and up-to-date. Since the Penterep SaaS use case must handle a large number of users and penetration testing can be resource-intensive, the cloud use case incorporates the following supporting services to ensure high availability, scalability, and stability of system operations: Kubernetes (The Kubernetes Authors, 2024), load balancing, backup and recovery, CI/CD pipelines, message queuing, and in-memory caching.

The second use case of the distribution of the Penterep environment is on-premise (OP). In this case, the environment is deployed on the customer's infrastructure, and the customer manages its operation. The penetration testing environment is deployed as a virtual machine (VM). In contrast to the cloud, each OP of the environment contains its static database. Originally, we assumed that the static database would not be part of the VM. However, since some customers conduct their testing in highly isolated environments without external network connectivity, we had to redesign the solution to include the static database directly within the VM. This redesign complicated the essential updates to static databases in VMs. To address this, we developed the following steps to keep the static database up-to-date: (i) the data are automatically backed up, (ii) the customer downloads the new version, (iii) the platform is re-deployed with the new version, and (iv) the data are restored to the updated version. From a technical standpoint, the supporting services available for cloud deployments are limited for on-premises VM use cases, requiring customers to configure their environment accordingly.

In both cases, the user interacts with the penetration testing environment through her/his web browser, which contains a modular Single-page application (SPA). Before the SPA is provided to the user, the user must first authenticate to the User Management Server (UMS). The UMS serves as an authentication and authorization server and contains all user accounts, licenses, and penetration testing modules. Once the user has successfully authenticated against the UMS, she/he is returned to the environment's application server with an assigned token from the UMS. The application server authenticates the token against the UMS, and if valid, the SPA is returned to the user and loaded into her/his web browser. From this point on, all requests are sent by the SPA based on user interaction and other application events. A single UMS is dedicated to the authentication process for all deployed environments (tenants and OPs), and single users can access multiple different environments if there is an existing invitation for them to do so (e.g., contract penetration testing for multiple companies running the environment).

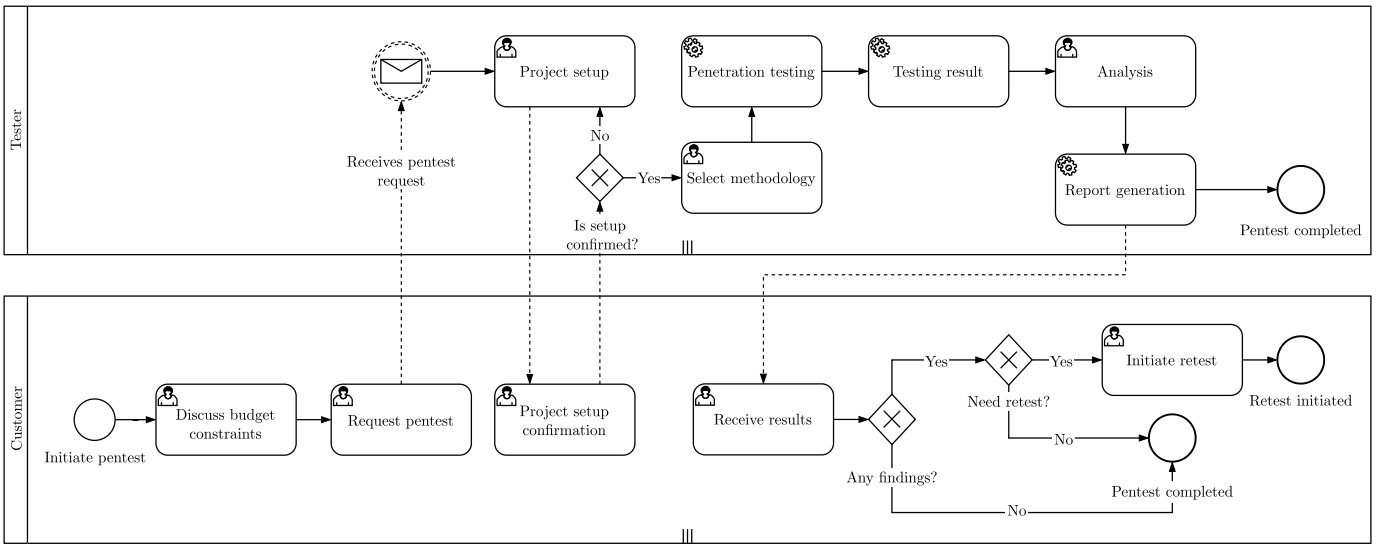


Fig. 4. Penetration testing workflow in the Penterep environment.

4.2. Penetration testing workflows

The penetration testing process in the Penterep environment consists of **five main phases**: 1. Project Setup, 2. Methodology Selection, 3. Penetration Testing, 4. Analysis, and 5. Report Generation. Each phase includes specific suboperations, as illustrated in Fig. 4.[2] Below is a workflow breakdown, focusing on the purpose, primary function, and incorporation of adaptable checklists.

1. Project setup

The penetration testing process begins when the customer initiates a test request. The following activities occur:

- **Discussion**: The customer and tester agree on project boundaries, budget constraints, and testing objectives.
- **Setup Confirmation**: A project is created, defining details such as contact information, testing scope, and deadlines.

The tester ensures that the setup aligns with the customer's requirements before proceeding to the next phase.

2. Methodology selection

The methodology phase focuses on creating a structured testing plan by selecting predefined **checklists** from the knowledge base. Each checklist corresponds to specific test targets, such as network devices (switches, routers, firewalls, etc.), servers (operating systems, databases, virtual machines), and web applications (APIs, content management systems, web services). Checklists ensure that all critical components are systematically tested. Each checklist contains: (i) a list of tests to be executed, (ii) detailed instructions to guide testers through each test, and (iii) predefined vulnerabilities linked to specific tests for consistency.

Multiple checklists can be combined, allowing testers to address complex environments. For example, testing an infrastructure with network devices, application servers, and running web applications can utilize a combination of three checklists. Once the methodology is confirmed, testers proceed to the penetration testing phase.

3. Penetration testing

The **penetration testing** phase involves executing the selected tests from the checklists:

- **Manual Testing**: Testers follow predefined instructions, verify findings, and document vulnerabilities. Each checklist ensures that no critical step is missed during penetration testing.
- **Automated Testing**: If possible, tests can be executed automatically. Automated tests are queued and run outside the Penterep environment, with results returned to the platform.

To minimize false positives, findings from automated tests are marked as *unverified* until validated by the tester. For every confirmed vulnerability, testers can: use predefined vulnerabilities from the knowledge base, which include descriptions, severity ratings, and remediation steps; modify or create new vulnerabilities as needed to align with the test results (e.g., very specific or zero-day vulnerability).

Overall, integrating checklists into the active testing process ensures consistency and accuracy, while detailed instructions support less experienced testers.

4. Analysis

In this phase, testers analyze and refine all findings identified during penetration testing. Key activities include:

- Reviewing vulnerabilities, validating automated testing results, and calculating severity.
- Providing additional comments and context for each vulnerability and other security flaws.

Checklists facilitate structured analysis by linking each test result to predefined vulnerabilities and remediation steps, ensuring clarity and completeness.

5. Report generation

The final phase involves generating a detailed penetration test report. The report includes:

- Description and parameters of the testing, including the target, scope, and other information defined in the project setup.
- A list of vulnerabilities with descriptions, location, category, severity, and recommended remediation.
- References to the checklists and tests performed, ensuring traceability of manual and automated testing.

[2] The penetration testing workflow diagram uses the Business Process Model and Notation (BPMN) 2.0 (Object Management Group, 2024).

- Attachments, such as logs or screenshots, to support findings.

The automated report generation significantly reduces the time testers spend on documentation. Reports are delivered securely to the customer. If a retest is required, testers use the same checklists to re-verify that vulnerabilities have been remediated, ensuring a consistent and reliable process.

4.3. Automated penetration testing

In the case of automated testing, SATs are deployed on the testers' devices (outside of the platform), giving them a full choice of where to place the SAT and control over it. This eliminates any limitations associated with testing on internal networks and the general use of the cloud service for offensive operations, which can be technically and legally problematic. To operate it, testers need to have a device with SAT installed, that will perform the testing and a testing manager that will pair with the platform and then request tests from the platform, execute them, and return the results for further processing, as can be seen in Fig. 5.

An automated test cannot be run freely and is only assigned to tests that could be automated (e.g., HTTP header check, URL query, etc.). The test cases and their parameters are predefined and only need to know the target of the test. This is taken from the project from which the test is loaded into the queue in a "scheduled" state. The SAT then requests tests from the queued ones from the platform in each released thread, changing the test state to "running". When the test is completed, the SAT sends the result back to the platform and the status changes to "finished".

Test results may vary. If a scan has been performed, the resources found are returned to the platform for further testing (e.g., devices, web pages, backup locations, etc.). If a vulnerability is found, it is associated with the specific test and the target that was tested. However, unlike manual penetration testing, the vulnerability status is set to "unverified" and such a finding must be verified. This is due to the risk of false positives, so testers must validate each automated test finding, or it will not be transferred to the report.

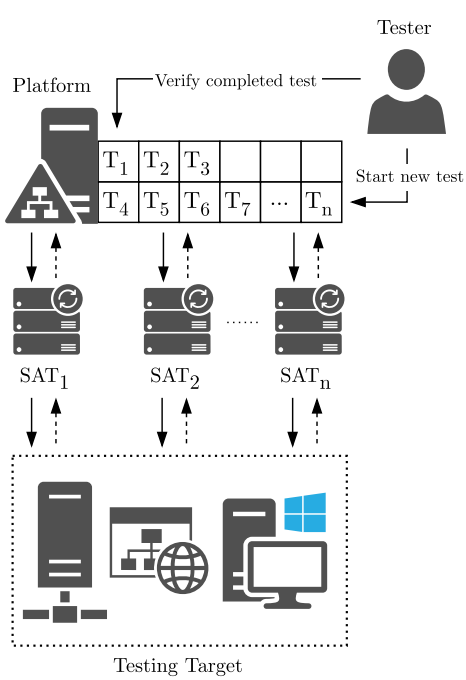


Fig. 5. Tester starts tests T_1, T_2, …, T_n that are added into queue from which all connected SAT_1, SAT_2, …, SAT_n request them. Independently, each SAT executes assigned tests against the specified target and then returns the results to the platform where the tester verifies them.

Individual test cases are created based on an automated tool with specific parameters that perform the test on the SAT. All supported automated testing tools are open source under the PenterepTools[3] toolkit and can also be used platform-independently. One of the main advantages of this approach is complete transparency, which is critical for penetration testing.

4.4. Benefits of the innovative approach

Based on the presented innovative approach and environment for penetration testing, the main benefits can be divided into completeness, accuracy, and transparency as follows:

- **Completeness**: Based on the agreed scope and limits of testing, checklists are selected during the creation of the penetration testing project that contains all the predefined tests that need to be performed. Until all tests are finished, the project cannot be considered complete. The final report is generated only from all completed tests and confirmed vulnerabilities to ensure completeness.
- **Accuracy**: Each test is created based on a methodology, standard, or from experts in the field. The tests are not generic and are divided into specific test cases, with one or more vulnerabilities associated with each test that can be detected by the test. If a test has been automated, it is marked as "unverified" and must be manually checked to avoid false positives.
- **Transparency**: Penetration tests are described in detail and referenced to their origin. In automated testing, the equipment performing the testing is under the full control of the tester. The automated test tools are open source and the output includes a log of all actions performed for possible revision.

In addition to the main inherent benefits listed above, we can also consider the benefits of the presented environment for penetration testing from the perspective of pentesters and customers, as described in the sections below.

4.4.1. Pentesters view

Considering the penetration tester's perspective, one of the main advantages of the presented penetration testing environment is the unification of all activities in one place with regard to teamwork. This is because, in general, if several testers work on one project, there is a risk of duplication, which is mitigated here by interactive checklists.

Another advantage is the inclusion of detailed tests that provide a structured approach along with practical examples, supporting both experienced and novice testers. While checklists alone may not cover every possible attack scenario, they serve as a valuable starting point by outlining key test cases and approaches. For less experienced testers, the detailed instructions offer guidance on initial attack vectors, though a deeper understanding of the testing context is still required. For senior testers, the checklist-based approach functions as an interactive checkpoint, ensuring that any critical steps are not overlooked during penetration testing.

In terms of time capability, one of the main benefits of the presented approach is the creation of a knowledge base that contains possible test targets and vulnerabilities in addition to the penetration tests. Similarly to a test, each vulnerability has a predefined description, severity, remediation recommendations, and other information that the tester can further specify in the context of the test objective. All of this, along with automatic report generation, significantly saves testers' time

[3] PenterepTools are freely available for download as open source under the GPLv3+ license at https://pypi.org/user/penterep/.

and eliminates the need to perform a significant amount of manual activities as would be the case with a branch-and-bound approach where testers keep notes in various applications (e.g., Excel, Word, Trello, etc.), while our penetration testing environment aggregates data and its management in one place.

4.4.2. Customer view

An important entity in the whole presented environment is also the customer, who has requested the penetration testing and after its completion reviews the final report. In our approach, we consider the presence of the customer directly in the project not only as one person but also as a group of customer employees (e.g., developers, system administrators, etc.) who have access to the individual test results, especially the identified vulnerabilities. After completion or during the actual test, they can fix the found vulnerabilities and request a retest. If necessary, it is also possible to have a discussion on individual findings directly with the penetration testers, thus facilitating communication between the two parties.

5. Case study

To validate our penetration testing environment, experimental testing was performed against a vulnerable web application OWASP Juice Shop (see Section 5.1) that contains selected vulnerabilities with severity of information, low, medium, high, and critical. Furthermore, to compare our approach and demonstrate the main advantages of our solution, the presented case study also includes the solutions compared in Section 2.2. Since the application is open source and well documented, we were able to measure the completeness and accuracy of the penetration testing within this case study. Completeness was measured by comparing the case study results with the vulnerabilities listed in the hacking guide on the official OWASP project website, where each challenge contains a specific vulnerability to be exploited (Kimminich, 2024). In addition, we also tested misconfigurations of the application that are not listed on the OWASP website but were required for verification by the used checklist. Accuracy was then assessed by reviewing all findings from the automated tests to include only true positive vulnerabilities listed in Table 3.

The vulnerable web application was deployed to a sandbox environment, which contained a tester that interacted with the Penterep environment to perform manual tests and run automated tests. Since in our approach the server for automated testing is always under the control of the tester, the sandbox environment also included the SAT connected to Penterep. In addition, Burp Suite, Acunetix, Nessus, w4af, and Wapiti were used in the case study. Regarding the initial setup, all automated tools were configured with the login credentials for the tested web application from the beginning of the testing, as a significant part of the application is accessible only to logged-in users.

After the testing was completed, the results from all tools were aggregated and thoroughly analyzed. Since we knew exactly what vulnerabilities the tested application contained, we limited the results from all tools to only the true positives. The entire environment and the steps of the experimental testing are shown in Fig. 6 and the case study parameters are listed in Table 2.

For the automated tests in the Penterep environment, the platform was connected to the SAT deployed on the tester's device, which included open source automated tools from the PenterepTools collection (PyPI, 2024), listed in Appendix B.

Table 2
Overview of penetration testing tools.

Platform	Version	License
Penterep	0.9.42	Enterprise
Burp Suite	2024.11	Professional
Acunetix	24.11.0	Premium
Nessus	10.7.6	Professional
w4af	1.6.49	GNU GPL v2.0
Wapiti	3.2.2	GNU GPL v2.0

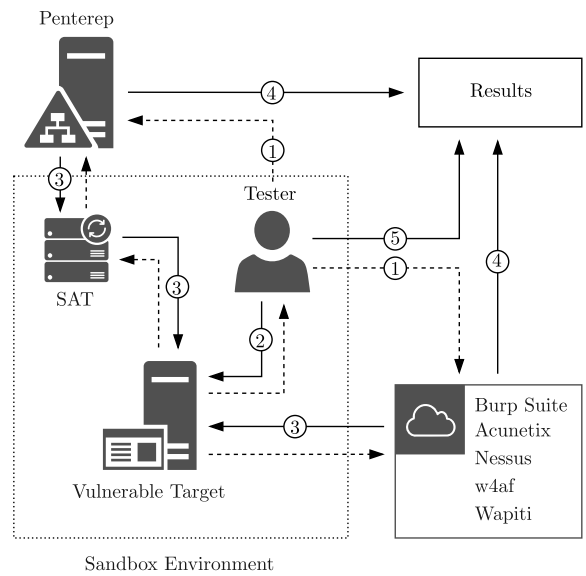


Fig. 6. Sandbox environment: 1. Interaction with tools and platform (checklists, guidelines, running automated tests, etc.); 2. Manual penetration testing; 3. Automated penetration testing; 4. Aggregation of all findings from penetration testing; 5. Results analysis filtering true positives.

5.1. Testing target

To verify our innovative approach, we chose the well-known open source vulnerable web application OWASP Juice Shop (version 17.1.1) (OWASP Foundation, 2024c). The application is written in Node.js and contains vulnerability categories, such as Broken Access Control, Cryptographic Failures, Injection, or Security Misconfiguration. Since penetration testing was limited to a sandbox environment and a single checklist, we purposely omitted Juice Shop challenges that were outside the scope of the case study, such as deriving a user's password from a social media profile or extracting hidden data from an image using steganography. The complete list of vulnerabilities found in this case study and their descriptions is provided in Appendix A.

5.2. Case study results

The test results in Table 3 show a significant limitation of restricting penetration testing to automated tools only. This is because some vulnerabilities require visual inspection and user interaction, represent business logic errors, or are located at a different place than the input, and their testing is problematic or impossible to automate. Moreover, the target in our case study was a SPA that renders its content based on client interaction, making manual testing necessary. This limitation is evident in the results from the Nessus, w4af and Wapiti, which were unable to test the SPA and identified only a limited number of vulnerabilities. This type of modern application poses significant challenges for test automation, as its structure is hidden by default. Without interaction or extensive application mapping, automated tools can achieve only a small or minimal coverage of the tested environment.

It is also important to note that if SAT or another automated tool is not used at all in combination with Penterep, completeness is not affected, as all tests can always be performed manually, while those that are automated are run optionally by the tester. In addition, each automated test has predefined parameters, making it single-purpose, which increases accuracy. Transparency is then ensured by the distribution of SAT tools as open source and running under full tester's control. The same applies to any tools that are under the tester's control, so transparency can be supported by solutions other than SAT, such as Burp Suite, which was included in the presented case study.

Table 3
Case study demonstrative results from penetration testing using Penterep and other tools Burp Suite, Acunetix, Nessus, w4af, and Wapiti, without additional plugins.

ID	Vulnerability	Severity	Penterep		Burp Suite	Acunetix	Nessus	w4af	Wapiti
			Manual	Automat					
01	API-only XSS	High	○	●	●	●	○	○	○
02	Access log	Medium	○	●	○	●	○	●	○
03	Admin registration	High	●	○	○	○	○	○	○
04	Allowlist bypass	Low	○	●	●	○	○	○	○
05	Arbitrary file write	High	●	○	○	○	○	○	○
06	Blocked RCE DoS	Medium	●	○	○	○	○	○	○
07	CAPTCHA bypass	Medium	●	○	○	○	○	○	○
08	Change password (CSRF)	High	●	○	○	○	○	○	○
09	Password strength	Low	●	○	○	○	○	○	○
10	Search query injection	Critical	○	●	○	●	○	○	○
11	CSP bypass	Medium	●	○	○	●	○	○	○
12	Client-side XSS	Medium	●	○	○	●	○	○	○
13	Confidential document	Low	○	●	○	○	○	○	○
14	User settings (CSRF)	High	●	○	○	○	○	○	○
15	DOM-based XSS	Medium	●	○	○	●	○	○	○
16	Database schema	Critical	●	○	○	○	○	○	○
17	Object modification	Medium	●	○	○	○	○	○	○
18	Cross-domain data	High	●	○	○	○	○	○	○
19	Error handling	Low	●	○	○	○	○	○	○
20	Exposed metrics	Low	○	●	○	●	○	○	○
21	Validation bypass	Medium	●	○	○	○	○	○	○
22	Forged signed JWT	High	●	○	○	○	○	○	○
23	Developer backup	Low	●	○	○	○	○	○	○
24	HTTP header XSS	Medium	●	○	○	○	○	○	○
25	Local file read	High	●	○	○	○	○	○	○
26	SQLi login	Critical	○	●	○	●	○	○	○
27	Payload manipulation	Medium	●	○	○	○	○	○	○
28	NoSQL DoS	High	●	○	○	○	○	○	○
29	NoSQL exfiltration	High	●	○	○	○	○	○	○
30	NoSQL manipulation	High	●	○	○	○	○	○	○
31	Payback manipulation	Medium	●	○	○	○	○	○	○
32	HTTP methods	Info	○	●	○	○	●	●	○
33	Reflected XSS	Medium	○	●	○	○	○	○	○
34	Repetitive registration	Low	●	○	○	○	○	○	○
35	SSRF attack	Medium	○	●	○	●	○	○	○
36	Template injection	High	●	○	○	○	○	○	○
37	Server-side XSS	High	○	●	●	●	○	○	○
38	Successful RCE DoS	High	●	○	○	○	○	○	○
39	Password length	Info	●	○	○	○	○	○	○
40	Upload size	Low	●	○	○	○	○	○	○
41	Upload type	Low	●	○	○	○	○	○	○
42	User credentials	Critical	○	●	○	○	○	○	○
43	Video XSS payload	Low	●	○	○	○	○	○	○
44	Session manipulation	Low	●	○	○	○	○	○	○
45	Vulnerable library	Medium	●	○	●	●	○	○	○
46	XXE data access	High	●	○	○	○	○	○	○
47	XXE DoS attack	High	●	○	○	○	○	○	○
48	HttpOnly cookie	Info	○	●	●	○	○	○	○
49	Secure cookie	Info	○	●	○	○	○	○	●
50	SameSite cookie	Info	○	●	●	○	○	○	○
51	CSP misconfiguration	Low	○	●	●	○	●	○	●
52	Robots.txt	Low	○	●	●	○	●	●	○
Discovered			35/52	17/52	8/52	11/52	3/52	3/52	2/52

The Penterep environment is built on checklists that must be selected before the start of penetration testing and thus represent a measurable parameter of test completeness. Therefore, this dependency can be perceived as a disadvantage from a certain perspective, since the tester always has to test according to a certain methodology, standard, and other procedures specified by the checklist. However, our approach is based on methodological testing, which is critical due to the nature of the security testing domain. This fact is underscored by the results of the presented case study, as without forcing a set of manual tests, it would be impossible to find all the vulnerabilities and other flaws present using only fully automated tools. Similarly, the automated tools within the Penterep environment would face substantial limitations in identifying vulnerabilities if used without the interactive checklists and manual testing.

5.3. Limitations

In this case study, penetration testing was performed on a single testing target. Although presenting results from multiple and different types of targets could expand the vulnerability list and better identify intersections between various tools, the primary objective of this paper is not to conduct an exhaustive survey of all penetration testing and scanning tools on a myriad of testing targets. Instead, the main focus is on introducing an innovative approach for comprehensive penetration testing that includes a combination of automated and manual tests verified on a demonstrative example.

Regarding the tested environment, another limitation was the inclusion of only a single custom checklist for web application penetration testing, which builds on the OWASP ASVS standard. As mentioned above, the main goal was not to conduct an evaluation but to demonstrate how the proposed approach can be applied in a practical setting.

Future research could address this by exploring various aspects of interactive checklists, such as focus, length, and combinations, to further examine their effectiveness in supporting penetration testing.

The findings in this case study present the vulnerabilities discovered by Penterep with Burp Suite, Acunetix, Nessus, w4af, and Wapiti. These tools allow for the integration of third-party plugins. Using these integrated plugins and their extensions, the tools have the potential to expand the list of identified vulnerabilities that could result in a significant overlap in the findings. However, our innovative approach, which focuses on adaptable interactive checklists, ensures that penetration testers do not overlook any critical test cases, including manual tests that are often neglected by other tools featured in the case study.

5.4. Lessons learned

For more efficient adoption of this new approach, we highlight the main lessons learned with the following general recommendations:

a *Scope the penetration testing properly:* Before starting penetration testing, it is important to determine the scope of the testing, the number of targets, support materials, deadlines, and any other testing conditions. To complete the test, measurable parameters must be established, which is supported by using checklists that determine what is to be tested against a given target, giving penetration testers and the customer a clear view of what has been tested and how.

b *Focus deeply on information gathering:* Information gathering aims to explore publicly available information about the target to identify the most effective methods for exploitation. This comprehensive process serves the purpose of understanding the functioning of the application or system, enabling the discovery of security vulnerabilities. This process involves two distinct approaches: Passive Information Gathering, which precedes active information gathering and is less invasive, relying solely on publicly available target information; and Active Information Gathering, which requires more preparation as it leaves traces, potentially triggering alerts to the target. This method involves actively engaging with the target, obtaining details about open ports, services, versions of operating systems and applications, and other pertinent information. A strong emphasis on information gathering provides crucial insights and intelligence, enabling penetration testers to conduct thorough and effective security assessments. In turn, this helps enhance the overall cybersecurity posture of the target system.

c *Analyze the results and verify the automated test results precisely:* Once penetration testing is conducted to identify vulnerabilities present in the given target, the tester has to precisely prioritize the vulnerabilities based on their potential impact and severity. Furthermore, the tester should perform a risk assessment that evaluates all true positive findings considering the potential damage and the value of the assets. Here, the tester has to precisely mark false positive results provided by automated tests to avoid misleading security alerts, which might lead to overlooking valid threats. Eliminating false positives provided by automated tests could be reduced by using Machine Learning (ML) (Ban et al., 2021; Alahmadi et al., 2022) or by tweaking the tool's settings to reduce the number of false positive claims.

d *Discuss the results with the clients and provide suitable recommendations:* The findings should be communicated to the clients through a presentation that emphasizes the identified issues along with the recommended mitigation measures. All of these discoveries should be discussed with the product owners and the development team to ensure that all stakeholders are informed. Furthermore, a comprehensive report should be provided detailing the list of vulnerabilities discovered with specific actions, configuration changes, or patches to improve overall system security. If needed, the penetration tester should also offer recommendations for ongoing improvements to promote security best practices, such as regular pentests or the integration of GitHub security checks (Bühlmann and Ghafari, 2022).

6. Conclusion

This paper introduces an innovative approach to conducting penetration tests through adaptable interactive checklists. These checklists provide crucial support to testers, ensuring that no essential aspects of testing are overlooked. The proposed approach serves as a valuable complement to established methodologies and standards such as PTES, OWASP, NIST, and ISO. The designed approach is developed in the Penterep environment demonstrating the innovative approach. In this paper, we also highlight the design of the Penterep environment including its deployment flexibility (both on-premise and cloud-based), and overview comparison with the existing tools following state-of-the-art penetration testing approaches. To the best of our knowledge, using our innovative approach, we bridge the gap between manual and automated approaches while ensuring flexibility, scalability, and effectiveness of penetration testing. The importance of emphasizing improvements in penetration testing is underscored by NIS2, which broadens the range of obligated entities and the scope of responsibilities. This includes the requirement to test and demonstrate cyber resilience (European Union, 2022).

Given the limitations of our penetration testing environment, we recognize open challenges for potential expansion to enhance the efficiency of penetration testing and its management. This involves incorporating third-party tools and emerging AI methods to strengthen the penetration testing process. Specifically, in our case, AI offers alternative avenues for extending our innovative approach by categorizing, classifying, and assigning suitable checklists during the initial penetration testing phases. Another open challenge pertains to supporting the initial non-technical step, the preparation phase. Here, AI could assign testers to specific tests and subsequently assess the confidence and accuracy of individual findings. Addressing these challenges has the potential to propel the current state-of-the-art forward and effectively broaden the scope of our innovative approach.

CRediT authorship contribution statement

Willi Lazarov: Writing – original draft, Software, Methodology, Conceptualization. **Pavel Seda:** Writing – original draft, Visualization, Data curation. **Zdenek Martinasek:** Supervision, Project administration, Funding acquisition. **Roman Kummel:** Validation, Resources, Investigation.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Willi Lazarov reports financial support was provided by Ministry of the Interior of the Czech Republic. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The research described in this paper was financially supported by the Ministry of the Interior of the Czech Republic, Czechia, project No. VK01030019. We would also like to express our gratitude to the National Cyber and Information Security Agency (NCISA) of the Czech Republic for testing and providing feedback on the presented approach and the Penterep environment.

Appendix A. Case study vulnerabilities

In the case study, a total of 52 vulnerabilities were found. The description and references to the Common Weakness Enumeration (CWE) and OWASP Top 10 categories (2021 version) of all present vulnerabilities are listed in Table A.4. All CWEs were referenced according to the MITRE database (MITRE Corporation, 2024).

Table A.4
Vulnerabilities found in the case study experimental testing.

ID	Vulnerability	CWE	OWASP	Description
01	API-only XSS	79	A03:2021	Missing or insufficient protection against XSS attacks
02	Access log	200	A01:2021	Allowed directory listing
03	Admin registration	20	A01:2021	Improper user input validation
04	Allowlist bypass	601	A01:2021	The user may be redirected to an untrusted page
05	Arbitrary file write	22	A01:2021	Missing or insufficient protection against Path Traversal
06	Blocked RCE DoS	502	A08:2021	Deserializes untrusted data without validation
07	CAPTCHA bypass	799	A04:2021	Broken or insufficient anti-spam protection
08	Change password (CSRF)	352	A01:2021	Missing or insufficient protection against CSRF attacks
09	Password strength	521	A07:2021	Users are allowed to use a weak password
10	SQLi (search)	89	A03:2021	Missing or insufficient protection against SQL injection attacks
11	CSP bypass	693	A05:2021	Missing or improper settings of Content Security Policy
12	Client-side XSS	79	A03:2021	Missing or insufficient protection against XSS attacks
13	Confidential document	200	A01:2021	Allowed directory listing
14	User settings (CSRF)	352	A01:2021	Missing or insufficient protection against CSRF attacks
15	DOM-based XSS	79	A03:2021	Missing or insufficient protection against XSS attacks
16	Database schema	89	A03:2021	Missing or insufficient protection against SQL injection attacks
17	Object modification	284	A01:2021	Allowed unauthorized direct access to objects
18	Cross-domain data	200	A01:2021	Exposure of sensitive information
19	Error handling	209	A05:2021	Error messages containing sensitive information
20	Exposed metrics	200	A01:2021	Exposure of sensitive information
21	Validation bypass	20	A03:2021	Improper user input validation
22	Forged signed JWT	1270	A01:2021	Manipulation or generation of incorrect security tokens
23	Developer backup	200	A01:2021	Ability to access backup files without authorization
24	HTTP header XSS	79	A03:2021	Missing or insufficient protection against XSS attacks
25	Local file read	200	A01:2021	Missing or insufficient protection against Local File Inclusion
26	SQLi login	89	A03:2021	Missing or insufficient protection against SQL injection attacks
27	Payload manipulation	639	A01:2021	Authorization bypass through user-controlled key
28	NoSQL DoS	943	A03:2021	Missing or insufficient protection against NoSQL injection attacks
29	NoSQL exfiltration	943	A03:2021	Missing or insufficient protection against NoSQL injection attacks
30	NoSQL manipulation	943	A03:2021	Missing or insufficient protection against NoSQL injection attacks
31	Payback manipulation	284	A01:2021	Allowed unauthorized direct access to objects
32	HTTP methods	749	A05:2021	Non-standard HTTP methods are allowed but not used
33	Reflected XSS	79	A03:2021	Missing or insufficient protection against XSS attacks
34	Repetitive registration	754	A08:2021	Improper check for unusual or exceptional conditions
35	SSRF attack	918	A10:2021	Missing or insufficient protection against SSRF attacks
36	Template injection	1336	A03:2021	Improper neutralization of special elements in a template engine
37	Server-side XSS	79	A03:2021	Missing or insufficient protection against XSS attacks
38	Successful RCE DoS	1333	A08:2021	Inefficient regular expression complexity
39	Password length	521	A07:2021	Users are not allowed to use passwords of 64 characters or more
40	Upload size	434	A05:2021	Unrestricted upload of file
41	Upload type	434	A05:2021	Unrestricted upload of file
42	User credentials	89	A03:2021	Missing or insufficient protection against SQL injection attacks
43	Video XSS payload	79	A03:2021	Missing or insufficient protection against XSS attacks
44	Session manipulation	639	A01:2021	Authorization bypass through user-controlled key
45	Vulnerable library	1395	A06:2021	Dependency on vulnerable third-party component
46	XXE data access	611	A05:2021	Improper restriction of XML External Entity reference
47	XXE DoS attack	611	A05:2021	Improper restriction of XML External Entity reference
48	HttpOnly cookie	1004	A05:2021	Sensitive cookie without "HttpOnly" flag
49	Secure cookie	614	A05:2021	Sensitive cookie in HTTPS session without "Secure" attribute
50	SameSite cookie	1275	A05:2021	Sensitive cookie with improper "SameSite" attribute
51	CSP misconfiguration	693	A05:2021	Missing or improper settings of Content Security Policy
52	Robots.txt	200	A01:2021	The content of the robots.txt file reveal confidential information

Appendix B. Case study tests

All vulnerabilities in the case study were found using the Penterep WASV checklist, which extends the OWASP ASVS 4.0.3 standard. The tests used to find the vulnerabilities are listed in Table B.5. Automated tests were performed with tools from the PenterepTools collection, listed in Table B.6.

Appendix C. Checklist-based penetration test

To illustrate how checklists integrate with open standards, we provide an example of a Penterep WAVS checklist test, PTL-WEB-INJECT-SQL, which extends OWASP ASVS 5.3.4 (v4.0.3). The ASVS test parameters, listed in Appendix C.1, include the code (identifier), description (test objective), assurance levels,[4] and a reference to the CWE. The extended WAVS test builds upon ASVS by incorporating additional, more detailed parameters, as outlined in Appendix C.2. These include code, objective, targets, description, instructions, linked vulnerabilities with predefined descriptions, and a reference to the OWASP ASVS

[4] OWASP ASVS assurance levels: Level 1 (*L1*) — Low assurance, fully testable without additional information (e.g., access to documentation). Level 2 (*L2*) — For applications handling sensitive data that require protection (most applications). Level 3 (*L3*) — For critical applications performing high-value transactions, providing important services, handling sensitive medical data, or requiring the highest level of trust.

Table B.5
Penetration tests used in the case study.

Test Code	Found Vulnerabilities	Description
PTL-WEB-DISCO-DICT	02	Identify new resources using a dictionary
PTL-WEB-INJECT-XSS	01, 12, 15, 24, 33, 37, 43	Test for unsafe output that allows XSS metacharacters
PTL-WEB-ACCESS-PARAM	03, 17, 21, 27, 31, 44	Test protection against unsafe parameter assignments
PTL-WEB-INJECT-REDIR	04	Test redirect or forwarding outside the allowed URLs
OWASP ASVS 12.3.2	05	Test misuse of file names to attack locally stored files
PTL-WEB-INJECT-EVAL	06	Test unsecured methods for parsing JSON with JavaScript
PTL-WEB-ACC-CAPONE	07	Test invalidation of CAPTCHA codes after its use
PTL-WEB-ACC-CSRF	08, 14	Test protection mechanisms against CRSF attacks
PTL-WEB-AUTH-PWMIN	09	Test enforcement of a minimum password length
PTL-WEB-INJECT-SQL	10, 16, 26, 42	Test SQL queries against SQL injections
PTL-WEB-MISCONF-CSP	11, 51	Verify proper configuration of the Content Security Policy
PTL-WEB-MISCONF-DIRLST	13	Check if the directory listing is disabled
PTL-WEB-API-RESTCALLB	18	Access data from another domain by adding a callback
PTL-WEB-INFO-ERRINF	19	Trigger error messages containing sensitive information
OWASP ASVS 4.2.1	20	Access sensitive data and APIs without authorization
OWASP ASVS 6.4.1	22	Verify the implementation of secure secret key management
OWASP ASVS 8.1.6	23	Check for the presence of backups stored in a unsafe place
OWASP ASVS 5.3.9	25	Test the application protection against LFI/RFI attacks
OWASP ASVS 5.3.4	28, 29, 30	Test parametrization of data selection or database queries
PTL-WEB-BLOGIC-TOCTOU	34	Check time to time of use (TOCTOU) issues
PTL-WEB-INJECT-SSRF	35	Test the application functions for SSRF attacks
PTL-WEB-INJECT-TEMPLATE	36	Test protection mechanisms against template injections
PTL-WEB-INJECT-CODE	38	Use the input to run the code on the server-side
PTL-WEB-AUTH-PWMAX	39	Test password maximum length (64 characters)
PTL-WEB-UPLOAD-MAXSIZE	40	Upload files bigger than application limit
PTL-WEB-UPLOAD-FILETYPE	41	Upload files with not allowed file types
PTL-WEB-SW-KNOWNVULN	45	Check presence of component with known vulnerabilities
PTL-WEB-INJECT-XXE	46, 47	Test protection mechanisms against XXE attacks
PTL-WEB-LSCOO-FLHTTP	48	Check cookie flag settings (attribute “HttpOnly”)
PTL-WEB-LSCOO-FLSEC	49	Check cookie flag settings (attribute “Secure”)
PTL-WEB-LSCOO-FLSAME	50	Check cookie flag settings (attribute “SameSite”)
PTL-WEB-MISCONF-ROBO	52	Check if robots.txt reveal confidential resources
PTL-WEB-HTTP-METHOD	32	Verify all HTTP methods used on the server

Table B.6
Automated tools used in the Penterep environment.

Tool	Version	Description
ptcookiechecker	0.0.2	Cookie security and configuration testing tool
ptcrossd	1.0.4	Testing tool for crossdomain.xml misconfigurations
pthost	1.0.4	Default vhost testing tool
ptinjector	0.0.2	Injection vulnerability testing tool
ptinsearcher	1.0.9	Website and file information extractor
ptmethods	1.0.3	HTTP methods testing tool
ptmultifinder	0.0.2	Testing of multiple domains using a wordlist
ptmultiviews	1.0.7	Apache multiviews detection and enumeration tool
ptnmap	0.0.3	Nmap wrapper for services scanning
ptprssi	1.0.2	Path-relative style sheet import testing tool
ptresheaders	0.0.6	Analyzing and testing HTTP response headers for security best
ptsamesite	1.0.3	Detect same-site scripting vulnerabilities across various domains
ptsecurixt	1.0.4	Auto locates security.txt files in their known locations
ptwebdiscover	1.0.8	Web source discovery tool (application crawling and mapping)

standard. The length of the test content may vary depending on the complexity of the specific test.

C.1. OWASP ASVS

Code: 5.3.4

Description: Verify that data selection or database queries (e.g., SQL, HQL, ORM, NoSQL) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from database injection attacks.

Levels: *L1*, *L2*, *L3*.

CWE: 89

C.2. Penterep WAVS

Code: PTL-WEB-INJECT-SQL

Objective: Test SQL queries against SQL injections.

Targets: User inputs, HTTP headers, URL query, Cookies, Session, Logs.

Description: The purpose of this test is to verify that all database queries (including inserting new records, deleting records, or modifying existing values) use parameterized queries, ORM, Entity Framework, or are otherwise protected from SQL injection attacks.

Instructions: Check whether it is not possible to corrupt the context of the original SQL query with specially modified input, and thus inject

your own code into the query. First, insert metacharacters like ’, " and \ into the tested input to see whether the application displays an incorrect query syntax error message when inserting these characters. This message would confirm the possibility of SQL injection. If the error message does not appear, try to insert logical operators to see whether the application’s output is the same for all tested values or whether it returns different results for the following format of URL query: `?parameter=value statement`, where statement could contain values such as “`and 1=1`”, “`and 1=0`”, “`or 1=1`”, “`and ’1’=’1`”, “`and ’1’=’0`”, “`and "1"="1`”, “`and "1"="0`” (for example “`article=53 and 1=0`” should not return any entity).

If you successfully identify a dangerous metacharacter and the application returns different results for logical truth than for logical false, the application’s behavior indicates that the input is vulnerable to a Boolean-based SQL injection. If a string is passed in the input, you can try to test the vulnerability to an SQL injection, for example, by splitting the string into multiple parts. If the tested application returns the same results without splitting the string or with splitting it but returns different results using a different string, this would indicate the presence of vulnerability. The merging string methods differ in individual database servers:

- MySQL: `‘foo’ ‘bar’`
- Microsoft: `‘foo’+‘bar’`
- Oracle and PostgreSQL: `‘foo’||‘bar’`

If a numeric value is expected in the input, you can also try to determine whether the user input is converted to a number. For example, if the application returns the same result when passing a number as when passing a number followed by any string but returns a different result when using a value of 0, or a number preceded by a random string, the behavior might indicate that the user input is first converted to a number before it is used in the query. Example of casting test:

- `article=53abc`
- `article=0`
- `article=abc53`

If the result of the database query does not affect the output, check the presence of susceptibility to a time-based SQL injection. Depending on the used database server, try injecting a query with a time-consuming operation or with a temporary suspending function. For the MySQL server, it is possible to use, for example, the `sleep()` function:

- `article=53 and sleep(5)`
- `article=53 or sleep(5)`
- `article=53’ or sleep(5) and ’1’=’1`
- `article=53" or sleep(5) and "1"="1`

Linked vulnerability: PTV-WEB-INJECT-SQL

- **Name**: Missing or insufficient protection against SQL injection attacks.
- **Category**: V5. Validation, Sanitization and Encoding.
- **Description**: During testing, it was discovered that the web application suffers from a deficiency in the form of missing or insufficient protection against SQL injection attacks. This vulnerability arises in situations where the application does not provide sufficient protection against the insertion of malicious SQL code into database queries.
- **Causes**: The cause of this deficiency may be negligence in implementing security measures, lack of awareness of SQL injection issues, and insufficient use of available protection mechanisms such as parameterized queries, ORMs (Object-Relational Mapping), or other techniques to protect against SQL injection attacks.
- **Manifestations**: The detected deficiency is manifested by the fact that the web application does not validate or detect user input, which is then directly used in SQL queries. This allows an attacker to insert malicious SQL code into queries that can lead to manipulation of database data, disclosure of sensitive information, or even execution of unauthorized database operations.
- **Impacts**: Exploitation of this flaw can lead to serious consequences, including leakage of sensitive data, breach of database integrity, disruption of application functionality, and compromise of an organization’s credibility and reputation. SQL injection attacks can allow an attacker to make unauthorized changes to the system, access sensitive information, or even control the entire application.
- **Recommendations**: We recommend taking immediate corrective action to address this deficiency. Adopting secure practices for creating and executing SQL queries is critical, including the use of parameterized queries, ORMs, or other techniques to safeguard against SQL injection. Proper validation and escaping of user inputs used in queries are essential to preventing such attacks. Additionally, it is crucial to regularly update and validate the security mechanisms and libraries utilized in the application to reduce the risk of SQL injection vulnerabilities.
- **CWE**: 89 (Improper Neutralization of Special Elements used in an SQL Command).
- **Severity (default)**: Medium.

References: OWASP ASVS 4.0 5.3.4

Data availability

Data will be made available on request.

References

Acunetix, 2024. Web application security scanner. https://www.acunetix.com/. (Accessed 20 January 2025).

Ahmed, N.B., Daclin, N., Olivaux, M., Dusserre, G., 2023. Cybersecurity challenges for field hospitals: impacts of emergency cyberthreats during emergency situations. Int. J. Emerg. Manag. 18, 274–292. http://dx.doi.org/10.1504/IJEM.2023.132387.

Alahmadi, B.A., Axon, L., Martinovic, I., 2022. 99% false positives: A qualitative study of SOC analysts’ perspectives on security alarms. In: 31st USENIX Security Symposium. USENIX Security 22, USENIX Association, Boston, MA, pp. 2783–2800, URL: https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi.

Alghamdi, A.A., 2021. Effective penetration testing report writing. In: 2021 International Conference on Electrical, Computer, Communications and Mechatronics Engineering. ICECCME, pp. 1–5. http://dx.doi.org/10.1109/ICECCME52200.2021.9591097.

Auricchio, N., Cappuccio, A., Caturano, F., Perrone, G., Romano, S.P., 2022. An automated approach to web offensive security. Comput. Commun. 195, 248–261. http://dx.doi.org/10.1016/j.comcom.2022.08.018, URL: https://www.sciencedirect.com/science/article/pii/S0140366422003267.

Aydos, M., Aldan, Ç., Coşkun, E., Soydan, A., 2022. Security testing of web applications: A systematic mapping of the literature. J. King Saud Univ. - Comput. Inf. Sci. 34 (9), 6775–6792. http://dx.doi.org/10.1016/j.jksuci.2021.09.018, URL: https://www.sciencedirect.com/science/article/pii/S131915782100269X.

Bacudio, A.G., Yuan, X., Chu, B.T.B., Jones, M., 2011. An overview of penetration testing. Int. J. Netw. Secur. Appl. 3 (6), 19.

Ban, T., Samuel, N., Takahashi, T., Inoue, D., 2021. Combat security alert fatigue with AI-assisted techniques. In: Proceedings of the 14th Cyber Security Experimentation and Test Workshop. CSET ’21, Association for Computing Machinery, New York, NY, USA, pp. 9–16. http://dx.doi.org/10.1145/3474718.3474723.

Bartusiak, A., Kühne, M., Nitschke, O., Lässig, J., Nicolai, S., Bretschneider, P., 2023. First step into automation of security assessment of critical infrastructures. Sustain. Energy Grids Netw. 36, 101139. http://dx.doi.org/10.1016/j.segan.2023.101139.

Bella, G., Biondi, P., Bognanni, S., Esposito, S., 2023. PETIoT: Penetration testing the internet of things. Internet Things 22, 100707. http://dx.doi.org/10.1016/j.iot.2023.100707, URL: https://www.sciencedirect.com/science/article/pii/S2542660523000306.

Blakley, B., Cranor, L., 2023. Katie moussouris: Vulnerability disclosure and security workforce development. IEEE Secur. Priv. 21 (1), 11–18. http://dx.doi.org/10.1109/MSEC.2022.3222043.

Bühlmann, N., Ghafari, M., 2022. How do developers deal with security issue reports on GitHub? In: Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing. SAC '22, Association for Computing Machinery, New York, NY, USA, pp. 1580–1589. http://dx.doi.org/10.1145/3477314.3507123.

Carey, M., Rogers, R., Criscuolo, P., Petruzzi, M., 2008. Nessus Network Auditing, Second Edition. Syngress, URL: https://books.google.cz/books?id=s4rUAQAACAAJ.

CIRT.net, 2024. Nikto 2.5. https://cirt.net/Nikto2. (Accessed 20 January 2025).

Dalalana Bertoglio, D., Zorzo, A.F., 2017. Overview and open issues on penetration test. J. Braz. Comput. Soc. 23 (1), 1–16.

Denis, M., Zena, C., Hayajneh, T., 2016. Penetration testing: Concepts, attack methods, and defense strategies. In: 2016 IEEE Long Island Systems, Applications and Technology Conference. LISAT, IEEE, pp. 1–6.

Devagiri, J., Sidike, P., Niyaz, Q., Yang, X., Smith, S., 2022. Augmented reality and artificial intelligence in industry: Trends, tools, and future challenges. Expert Syst. Appl. 207, 118002. http://dx.doi.org/10.1016/j.eswa.2022.118002.

El Kafhali, S., El Mir, I., Hanini, M., 2022. Security threats, defense mechanisms, challenges, and future directions in cloud computing. Arch. Comput. Methods Eng. 29, 223–246. http://dx.doi.org/10.1007/s11831-021-09573-y.

European Union, 2022. Directive (EU) 2022/2555 of the european parliament and of the council. https://eur-lex.europa.eu/eli/dir/2022/2555. (Accessed 20 January 2025).

Gangupantulu, R., Cody, T., Park, P., Rahman, A., Eisenbeiser, L., Radke, D., Clark, R., Redino, C., 2022. Using cyber terrain in reinforcement learning for penetration testing. In: 2022 IEEE International Conference on Omni-layer Intelligent Systems. COINS, pp. 1–8. http://dx.doi.org/10.1109/COINS54846.2022.9855011.

GitHub, 2024. Wapiti – Web vulnerability scanner. https://github.com/wapiti-scanner/wapiti. (Accessed 20 January 2025).

Heiding, F., Süren, E., Olegård, J., Lagerström, R., 2023. Penetration testing of connected households. Comput. Secur. 126, 103067. http://dx.doi.org/10.1016/j.cose.2022.103067, URL: https://www.sciencedirect.com/science/article/pii/S016740482200459X.

ISECOM, 2024. Research. https://www.isecom.org/research.html. (Accessed 20 January 2025).

ISO, 2024. ISO/IEC 27001:2022. https://www.iso.org/standard/27001. (Accessed 20 January 2025).

Jayadeva, S., Ahmed, A.A.A., Malik, R., Shaikh, A.A., Siddique, M., Naved, M., 2023. Roles of cloud computing and internet of things in marketing management: A critical review and future trends. In: Proceedings of Second International Conference in Mechanical and Energy Technology. pp. 165–173. http://dx.doi.org/10.1007/978-981-19-0108-9_18.

Kimminich, B., 2024. Part II - Challenge hunting. https://help.owasp-juice.shop/part2/. (Accessed 20 January 2025).

McKinnel, D.R., Dargahi, T., Dehghantanha, A., Choo, K.K.R., 2019. A systematic literature review and meta-analysis on artificial intelligence in penetration testing and vulnerability assessment. Comput. Electr. Eng. 75, 175–188.

Mishra, S., 2021. Efficacy of unconventional penetration testing practices. Intell. Autom. Soft Comput. 31, 223–239. http://dx.doi.org/10.32604/iasc.2022.019485.

MITRE Corporation, 2024. CWE – Common weakness enumeration. https://cwe.mitre.org/. (Accessed 20 January 2025).

Muthuppalaniappan, M., Stevenson, K., 2020. Healthcare cyber-attacks and the COVID-19 pandemic: An urgent threat to global health. Int. J. Qual. Heal. Care 33, http://dx.doi.org/10.1093/intqhc/mzaa117.

Nidhra, S., Dondeti, J., 2012. Black box and white box testing techniques-a literature review. Int. J. Embed. Syst. Appl. (IJESA) 2 (2), 29–50.

NIST, 2020. Security and Privacy Controls for Information Systems and Organizations. Technical Report NIST Special Publication 800-53, Revision 5, U.S. Department of Commerce, Washington, D.C., URL: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf.

NMAP.org, 2025. Nmap. https://nmap.org/. (Accessed 20 January 2025).

Object Management Group, 2024. Business process model and notation (BPMN), version 2.0. https://www.omg.org/spec/BPMN/2.0/PDF. (Accessed 20 January 2025).

OWASP Foundation, 2024a. OWASP application security verification standard. https://owasp.org/www-project-application-security-verification-standard/. (Accessed 20 January 2025).

OWASP Foundation, 2024b. OWASP integration standards. https://owasp.org/www-project-integration-standards/. (Accessed 20 January 2025).

OWASP Foundation, 2024c. OWASP juice shop. https://github.com/juice-shop/juice-shop. (Accessed 20 January 2025).

PortSwigger, 2025. Burp suite. https://portswigger.net/burp. (Accessed 20 January 2025).

PTES, 2024. High level organization of the standard. http://www.pentest-standard.org/. (Accessed 20 January 2025).

PyPI, 2024. Penterep. https://pypi.org/user/penterep/. (Accessed 20 January 2025).

Rak, M., Salzillo, G., Granata, D., 2022. ESSecA: An automated expert system for threat modelling and penetration testing for IoT ecosystems. Comput. Electr. Eng. 99, 107721.

RAPID metasploit, 2025. Metasploit. https://www.metasploit.com/. (Accessed 20 January 2025).

Rathore, B., Brunner, M., Dilaj, M., Herrera, O., Brunati, P., Subramaniam, R.K., Raman, S., Chavan, U., 2024. Information system security assessment framework. https://untrustednetwork.net/files/issaf0.2.1.pdf. (Accessed 20 January 2025).

Rawal, B., Manogaran, G., Peter, A., 2023. The basics of hacking and penetration testing. In: The Basics of Hacking and Penetration Testing. pp. 21–46. http://dx.doi.org/10.1007/978-981-19-2658-7_2.

Scarfone, K., Souppaya, M., Cody, A., Orebaugh, A., 2008. Technical guide to information security testing and assessment. NIST Spec. Publ. 800 (115), 2–25.

Sharif, M.H., Mohammed, M., 2022. A literature review of financial losses statistics for cyber security and future trend. World J. Adv. Res. Rev. 15, 138–156. http://dx.doi.org/10.30574/wjarr.2022.15.1.0573.

Shulha, O., Ianenkova, I., Kuzub, M., Muda, I., Nazarenko, V., 2022. Banking information resource cybersecurity system modeling. J. Open Innov. Technol. Mark. Complex. 8, http://dx.doi.org/10.3390/joitmc8020080.

SQLmap, 2024. Automatic SQL injection and database takeover tool. https://sqlmap.org/. (Accessed 20 January 2025).

Tadiboina, S.N., 2022. The integration of handheld and smartphone-connected technologies into the doctor-patient relationship-AI. J. Posit. Sch. Psychol. 6 (11), 2933–2940.

Teichmann, F., Boticiu, S., 2023. An overview of the benefits, challenges, and legal aspects of penetration testing and red teaming. Int. Cybersecur. Law Rev. 4, 1–11. http://dx.doi.org/10.1365/s43439-023-00100-2.

Tenable, 2024. Nessus vulnerability scanner: Network security solution. https://www.tenable.com/products/nessus. (Accessed 20 January 2025).

The Kubernetes Authors, 2024. Kubernetes: Production-grade container orchestration. https://kubernetes.io. (Accessed 20 January 2025).

Trickel, E., Pagani, F., Zhu, C., Dresel, L., Vigna, G., Kruegel, C., Wang, R., Bao, T., Shoshitaishvili, Y., Doupé, A., 2023. Toss a fault to your witcher: Applying grey-box coverage-guided mutational fuzzing to detect SQL and command injection vulnerabilities. In: 2023 IEEE Symposium on Security and Privacy. SP, pp. 2658–2675. http://dx.doi.org/10.1109/SP46215.2023.10179317.

Vats, P., Mandot, M., Gosain, A., 2020. A comprehensive literature review of penetration testing & its applications. In: 2020 8th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions). ICRITO, IEEE, pp. 674–680. http://dx.doi.org/10.1109/ICRITO48877.2020.9197961.

W4af, 2024. Open source web application security scanner. https://w4af.readthedocs.io/en/latest/. (Accessed 20 January 2025).

Wasserman, L., Wasserman, Y., 2022. Hospital cybersecurity risks and gaps: Review (for the non-cyber professional). Front. Digit. Heal. 4, 862221. http://dx.doi.org/10.3389/fdgth.2022.862221.

Xie, B., Li, Q., Qian, H., 2022. Weak password scanning system for penetration testing. In: Meng, W., Conti, M. (Eds.), Cyberspace Safety and Security. Springer International Publishing, Cham, pp. 120–130.

ZAP, 2025. Zed Attack Proxy (ZAP). https://www.zaproxy.org/. (Accessed 20 January 2025).

Zhu, Y., Cui, L., Ding, Z., Li, L., Liu, Y., Hao, Z., 2022. Black box attack and network intrusion detection using machine learning for malicious traffic. Comput. Secur. 123, 102922. http://dx.doi.org/10.1016/j.cose.2022.102922, URL: https://www.sciencedirect.com/science/article/pii/S0167404822003145.

Willi Lazarov is a researcher at the Faculty of Electrical Engineering and Communication, Brno University of Technology (BUT), where he specializes primarily in research on profiling, interactivity, and active feedback methods to improve the effectiveness of cybersecurity education. As a Co-PI of security research projects, he also focuses on penetration testing and source code security analysis. He received his MSc at BUT in 2024, which he followed up with doctoral studies. He also actively cooperates with the Czech Army, National Cyber and Information Security Agency, University of Defence, and research groups at Tampere University in Finland and Pennsylvania State University in the US. In recognition of his contributions, he has received awards such as the EY Cyber Security Trophy, EY ESO Cyber Security Future Promise, BUT Rector's Award, and Minister of the Interior Award for outstanding achievements in security research.

Pavel Seda is a postdoctoral researcher at Brno University of Technology (BUT), specializing in cybersecurity and heuristic algorithms. He earned his MSc degree in 2017 and later received a PhD in 2022 from the Faculty of Electrical Engineering and Communication at BUT. In addition, he earned an MSc and Advanced MSc in Computer Systems, Communication, and Security from Masaryk University (MU), Faculty of Informatics in 2018 and 2023. Pavel's professional experience includes working as a Java developer at IBM Czech Republic from 2014 to 2018, serving as a Software Engineer at MU from 2017 to 2022, and various other professional experiences.

Zdenek Martinasek is an Associate Professor at the Faculty of Electrical Engineering and Communication at Brno University of Technology (BUT), Czechia. He completed his MSc degree in 2008 and received his PhD degree in 2013. His research activities focus on applied cryptography, side-channel cryptanalysis, and cybersecurity. His biggest expertise is in power analysis attacks based on machine learning. Currently, he is involved as a task leader in security research projects of the Ministry of Interior of the Czech Republic. He has published more than 45 papers in international journals and conferences and has provided several invited research and teaching lectures focused on cybersecurity. During his PhD studies, he completed an internship at TU Wien in Austria. He also helps to cover pedagogically Bachelor's and Master's program courses in Information Security.

Roman Kummel currently serves as the CEO of HACKER Consulting Ltd. Throughout his career, he held positions as an IT administrator in various companies from 1994 to 2011. Later, he transitioned to Seznam.cz, where he worked as a penetration tester specializing in web applications. Building on this experience, Roman began sharing his knowledge with the cybersecurity community through lectures at GOPAS. In the course of his teaching endeavors, he has educated over a thousand cybersecurity professionals, solidifying his recognition within the industry. Additionally, he holds industry-recognized certifications in Ethical Hacking, including CEHv11. Roman has also made significant contributions to the field, discovered vulnerabilities such as CVE-2019-2089 in the Android operating system, and developed innovative offensive techniques like File From Frame Hijacking.